

## 1\_Image\_classification\_CIFAR10\_CNN(lightning)

```
transform = transforms.Compose( # transform is from torchvision (only for image)
    [transforms.ToTensor(), # image to tensor --> divide by 255
     transforms.Resize((32, 32))])

batch_size = 32
```

ก่อนจะนำ Image เข้าไป Train จะต้องทำการแปลง Format ของรูปภาพก่อนโดยแปลงให้เป็น Type Tensor

แล้ว Tensor จะทำการ Normalize ค่าสีของแต่ละ pixel แต่ละจุดให้เป็น 0 ถึง 1 โดยการหาร 255

```
trainvalset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
trainset, valset = torch.utils.data.random_split(trainvalset, [40000, 10000])

trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)
valloader = torch.utils.data.DataLoader(valset, batch_size=batch_size, shuffle=False)

testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle=False)

#classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck')

100% | ██████████ | 170M/170M [00:12<00:00, 13.3MB/s]
```

กำหนดให้แบ่ง train set กับ validation set เป็นจำนวน 40000 ต่อ 10000 จากนั้น Dataloader จะส่ง Data เข้าไป train ต่อ 1 Iteration เป็นจำนวน 32 รูป

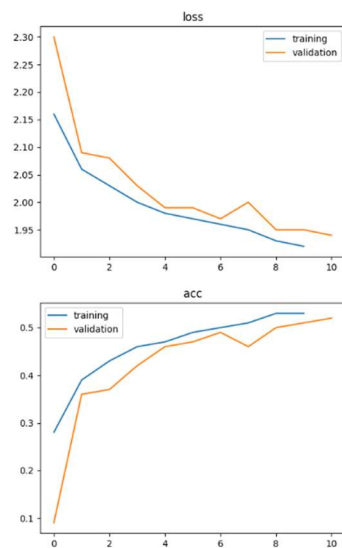
```
class CNN(pl.LightningModule):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5) # 3 input channels, 6 output channels, 5*5 kernel size
        self.pool = nn.MaxPool2d(2, 2) # 2*2 kernel size, 2 strides
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(400, 120) # dense input 400 (16*5), output 120

        self.fc2 = nn.Linear(120, 84) # dense input 120, output 84
        self.fc3 = nn.Linear(84, 10) # dense input 84, output 10
        self.softmax = torch.nn.Softmax(dim=1) # perform softmax at dim[1] (batch,class)

        self.criterion = nn.CrossEntropyLoss()
        self.accuracy = Accuracy(task="multiclass", num_classes=10)
        self.train_loss = []
        self.train_acc = []
        self.val_loss = []
        self.val_acc = []
        self.train_metrics_epoch = []
        self.val_metrics_epoch = []
        self.n = 0
```

Define layer ของ CNN โดยที่จะมีการ Define Layer แรกให้เป็น Convolutional 2d โดยกำหนด Input เป็น 3 ซึ่งก็คือ R, G, B ที่ Normalize มาแล้ว โดย Output size =  $(32 + 2 \times 0 - 5)/1 + 1 = 28$  ทำให้ขนาดของ sample เป็น 28x28 และมีจำนวน feature map เป็น 6 และจะไปเข้า MaxPool2d เพื่อลดขนาดของภาพลงครึ่งหนึ่ง เพราะ กำหนด Strid และ Padding เป็น 2 ทำให้ขนาด Output size หลังผ่านเป็น  $(28 + 2 \times (0) - 2)/2 + 1 = 14 \times 14$  และไปเข้า Fully connected ต่อไป และผลจาก Layer 2 จะได้ output size เป็น 10x10 แล้ว 16 feature map ซึ่งไปผ่าน Fully connected อีกตามรูปด้านล่างจะได้ว่า output size เป็น 5x5 ซึ่งเอามา flatten จะได้เป็น  $16 \times 5 \times 5 = 400$  และกำหนดเลือก Feature มา 120 feature แล้วไปเข้า Relu เพื่อทำให้เป็น non-linear จนสุดท้ายก็ไปเข้า Softmax ซึ่ง input ก่อนเข้า Softmax เป็น 10 ซึ่งเท่ากับจำนวนคลาส แล้วไป Normalize ด้วย softmax เพื่อเลือกคำตอบที่ดีที่สุด

```
def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = torch.flatten(x, start_dim=1) # flatten all dimensions (dim[1]) except batch (dim[0])
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
    x = self.softmax(x)
    return x
```



ซึ่งก็คือ ผลลัพธ์ของ model จะเห็นว่า model ได้ผลความแม่นยำเป็น 0.5 กับ validation set ซึ่งดีกว่า การเดาสุ่ม 10 classes ที่คิดเป็น 1/10 หรือ 0.1

```
testing ...
... 100%  313/313 [00:01<00:00, 178.59it/s]
testing loss: 1.938
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.5934    | 0.5180 | 0.5531   | 1000    |
| 1            | 0.6777    | 0.5950 | 0.6337   | 1000    |
| 2            | 0.4368    | 0.3250 | 0.3727   | 1000    |
| 3            | 0.3442    | 0.3490 | 0.3466   | 1000    |
| 4            | 0.4292    | 0.4880 | 0.4567   | 1000    |
| 5            | 0.5257    | 0.3270 | 0.4032   | 1000    |
| 6            | 0.5298    | 0.6580 | 0.5870   | 1000    |
| 7            | 0.5856    | 0.6090 | 0.5971   | 1000    |
| 8            | 0.5629    | 0.6980 | 0.6232   | 1000    |
| 9            | 0.5190    | 0.6280 | 0.5683   | 1000    |
| accuracy     |           |        | 0.5195   | 10000   |
| macro avg    | 0.5204    | 0.5195 | 0.5142   | 10000   |
| weighted avg | 0.5204    | 0.5195 | 0.5142   | 10000   |

เมื่อเอามาทดสอบกับ Test set ได้ผลดังข้างต้นจะเห็นว่า class 3 มีค่า f1\_score, precision, recall ต่ำกว่าทุก class เห็นได้ชัด แสดงให้เห็นว่า Class 3 ยากในการทำนาย และการที่ precision ต่ำแสดงว่า พยากรณ์ Class 3 ได้ถูกจริงๆเมื่อเทียบกับการ พยากรณ์ class 3 ทั้งหมดนั้นต่ำ ส่วน recall ต่ำแสดงว่า การพยากรณ์เป็น class 3 จริงๆเทียบกับ จำนวน class 3 จริงๆนั้นต่ำ

## 2.1\_Image\_classification\_Animal\_EfficientNetV2(lightning)

```
import albumentations as A
from albumentations.pytorch import ToTensorV2

transform_train = transforms.Compose([
    transforms.Resize((230,230)),
    transforms.RandomRotation(30),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.507, 0.487, 0.441], std=[0.267, 0.256, 0.276]) #nomalize imagenet pretrain
])

transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.507, 0.487, 0.441], std=[0.267, 0.256, 0.276])
])

batch_size = 32
```

โค้ดข้างต้นเป็นการทำ Data augmentation ที่มีการแบ่งแยกการ Transform data ที่แบ่งแบ่งของ train กับ test โดยที่ train จะมีการทำ สุ่มหมุน 30 องศาและทำ ทำการสุ่ม crop ภาพให้เป็นขนาด 224 x 224 และทำการ flip ตามแกน x,y ตามลำดับ และแปลง เป็น tensor และจบด้วยการ Normalize ค่าสีตามค่าที่กำหนด ซึ่งค่าเหล่านั้นเป็นค่าเฉลี่ยที่ได้มาจาก เฉลี่ยค่าสี RGB ของรูปภาพในโลก สาเหตุที่ต้องทำทั้งหมดนี้เพื่อให้ข้อมูลในการ train มีความหลากหลายมากขึ้น ไม่ได้เป็นภาพตรงๆเหมือนเดิม

ส่วน transform ของ test จะเป็นแค่การแปลงเป็น tensor และ Normalize ค่า RGB เท่านั้นเพราะ ตอนทดสอบเรา ห้ามเปลี่ยนแปลงข้อมูลของรูปภาพ และกำหนด Batch size ต่อ 1 iteration เป็น 32

```
trainset = AnimalDataset('./Dataset_animal2/train',transform_train)
valset = AnimalDataset('./Dataset_animal2/val',transform)
testset = AnimalDataset('./Dataset_animal2/test',transform)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)
valloader = torch.utils.data.DataLoader(valset, batch_size=batch_size, shuffle=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle=True)
```

ทำการสร้าง dataloader ของ train, test, validation แต่สิ่งที่น่าจะไม่ถูกก็คือ การ shuffle validation set กับ test set เพราะยากในการ Optimize เพราะ data ที่เข้าในการ validation จะต่างกันทำให้ผลลัพธ์ในแต่ละครั้งต่างกัน

```

class LitEfficientNetV2(pl.LightningModule):
    def __init__(self, num_classes=10, learning_rate=1e-3):
        super().__init__()
        # Load EfficientNetV2 model from torchvision
        self.model = models.efficientnet_v2_s(weights="IMAGENET1K_V1")
        # Replace the classifier with a custom layer for our task
        self.model.classifier[1] = nn.Linear(self.model.classifier[1].in_features, num_classes)
        self.criterion = nn.CrossEntropyLoss()
        self.accuracy = Accuracy(task="multiclass", num_classes=num_classes)
        self.learning_rate = learning_rate
        self.train_metrics = []
        self.val_metrics = []

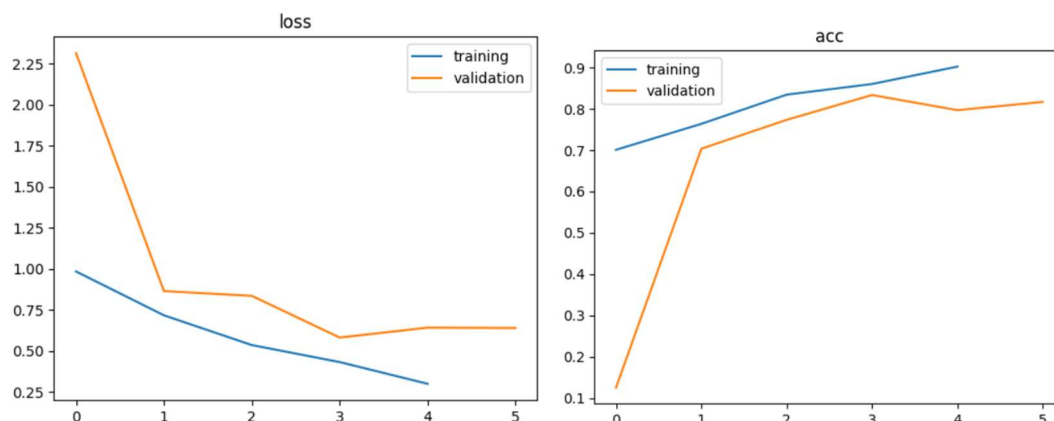
    def forward(self, x):
        return self.model(x)

```

จากโค้ดข้างต้นเราจะเห็นว่าเราไม่ได้ Defined Layer ด้วยตัวเองแล้ว แต่เป็นการไปใช้ Model ที่ Define layer แบบสำเร็จรูปมาให้แล้วซึ่งก็คือ model efficientnet\_v2 โดยการหนด weight เริ่มต้นเป็น weight ที่ model เคยถูก train ไว้แล้ว แล้วก็กำหนดให้ Classifier ทำการจำแนก เฉพาะ 10 class ของเราเท่านั้น ซึ่งสิ่งที่เรากำลังทำ คือ เตรียมการ Finetuning และ ก็มีการใช้ Crossentropy เป็น lossfunction

| Layer                     | Size               | Count      |
|---------------------------|--------------------|------------|
| LitEfficientNetV2         | [32, 10]           | --         |
| EfficientNet: 1-1         | [32, 10]           | --         |
| Sequential: 2-1           | [32, 1280, 7, 7]   | --         |
| Conv2dNormActivation: 3-1 | [32, 24, 112, 112] | 696        |
| Sequential: 3-2           | [32, 24, 112, 112] | 10,464     |
| Sequential: 3-3           | [32, 48, 56, 56]   | 303,552    |
| Sequential: 3-4           | [32, 64, 28, 28]   | 589,184    |
| Sequential: 3-5           | [32, 128, 14, 14]  | 917,680    |
| Sequential: 3-6           | [32, 160, 14, 14]  | 3,463,840  |
| Sequential: 3-7           | [32, 256, 7, 7]    | 14,561,832 |
| Conv2dNormActivation: 3-8 | [32, 1280, 7, 7]   | 330,240    |
| AdaptiveAvgPool2d: 2-2    | [32, 1280, 1, 1]   | --         |
| Sequential: 2-3           | [32, 10]           | --         |
| Dropout: 3-9              | [32, 1280]         | --         |
| Linear: 3-10              | [32, 10]           | 12,810     |

ซึ่งนี่ก็คือ layer ของ Efficientnet v2 ซึ่งจะเห็นว่าจำนวน Parametr สูงสุดคือ 14 ล้านกว่า parameter



จะเห็นว่า Loss น้อยกว่า CNN ที่เรา Defined Layer เองในตอนแรก และ accuracy ก็พุ่งมาถึง 0.9 จากการ Finetuning ด้วย dataset ของเรามากขึ้น

```
... testing ...
บเอดต์พุดของเซลล์โค้ด 10/10 [00:01<00:00, 5.54it/s]
testing loss: 0.5074
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.8571    | 0.8000 | 0.8276   | 30      |
| 1            | 0.9000    | 0.9000 | 0.9000   | 30      |
| 2            | 0.8000    | 0.9333 | 0.8615   | 30      |
| 3            | 0.9333    | 0.4667 | 0.6222   | 30      |
| 4            | 0.7931    | 0.7667 | 0.7797   | 30      |
| 5            | 0.8438    | 0.9000 | 0.8710   | 30      |
| 6            | 0.6444    | 0.9667 | 0.7733   | 30      |
| 7            | 0.9167    | 0.7333 | 0.8148   | 30      |
| 8            | 0.8710    | 0.9000 | 0.8852   | 30      |
| 9            | 0.9032    | 0.9333 | 0.9180   | 30      |
| accuracy     |           |        | 0.8300   | 300     |
| macro avg    | 0.8463    | 0.8300 | 0.8253   | 300     |
| weighted avg | 0.8463    | 0.8300 | 0.8253   | 300     |

จะเห็นว่า macro avg f1\_score พุ่งขึ้นไปถึง 0.8253 สูงกว่ารอบที่แล้วเกือบ 0.3 อีกทั้ง class 3 ที่เคยทำนายได้ไม่ดี ก็พุ่งขึ้นมาจาก 0.3 กว่าๆเป็น 0.622 โดยที่ precision สูงมากๆ ทำให้สามารถสรุปได้ว่า ทำนายว่าเป็น class 3 จริงๆแล้วทายถูกคิดเป็น 93.33% แต่ recall ก็ยังคงไม่ถึง 0.5 ทำให้เราเห็นว่า class 3 มีการจำแนกยากพอสมควร อาจจะหารายละเอียดที่เหมือนกันได้ยาก เลยทำนายว่าเป็น class 3 เมื่อเทียบกับ class 3 จริงๆแค่ 46.67% และ model สับสนระหว่าง class 3 กับ 6 ซึ่งก็คือ วัวกับม้าจะเห็นว่าทำนายว่าเป็น ม้าทั้งๆที่เป็น วัว ถึง 14 ครั้งจากทั้งหมด 28 ครั้งที่เป็นวัว



## 2.2 Image\_classification\_Animal\_EfficientNet(lightning)\_wandb\_HuggingFace

```
from albumentations.pytorch import ToTensorV2

transform_train = transforms.Compose([
    transforms.Resize((230,230)),
    transforms.RandomRotation(30),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.507, 0.487, 0.441], std=[0.267, 0.256, 0.276]) #nomalize imagenet pretrain
])

transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.507, 0.487, 0.441], std=[0.267, 0.256, 0.276])
])

batch_size = 32
```

สร้าง transformation data โดยแบ่งเป็น train กับ test โดย train จะมีการ resize เป็น 230,230 และทำการ random rotation หมุนมันสามสิบองศา และ flip แนวแกน x และ y ตามลำดับจาก นั้นแปลงเป็น tensor type เพื่อประมวลผลเร็วขึ้นและทำการ normalize ค่าสีด้วยค่าคงที่ที่ค่านี้ที่ได้มาจาก ค่าเฉลี่ยสีของรูปบน Internet และ test จะทำการ Resize 224x224 และเป็นเป็น tensor และ normalize ค่าสีเท่านี้สาเหตุที่เป็นแบบนี้ เพราะ data ที่ใช้ test ห้ามถูกเปลี่ยนแปลง หรือแก้ไขซึ่งต่างจากการ train ที่การปรับหมุนรูป data หรือที่เรียกว่า การทำ transform data ทำให้ข้อมูลมีความหลากหลายมากขึ้น ทำให้ model ได้เรียนรู้จากข้อมูลใหม่ๆ ที่ใช้ข้อมูล เดิมๆได้

```
trainset = AnimalDataset('./Dataset_animal2/train',transform_train)
valset = AnimalDataset('./Dataset_animal2/val',transform)
testset = AnimalDataset('./Dataset_animal2/test',transform)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)
valloader = torch.utils.data.DataLoader(valset, batch_size=batch_size, shuffle=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle=True)
```

สร้าง trainset, validation set, test set เพื่อนำไปใช้ในการสร้าง loader โดย กำหนด batch size เป็น 32 ทำให้ 1 iteration มีรูปภาพ 32 รูปแต่สิ่งที่แปลกและน่าจะมีคือ validation กับ test loader ไม่ควรต้อง shuffle เพราะทำให้ evaluate ยากเพราะผลแต่ละครั้งจาก loader จะเปลี่ยนไป ทำให้บางครั้ง test กับ validation set ดีแต่บางครั้งก็ไม่ได้

```

class LitEfficientNetV2(pl.LightningModule):
    def __init__(self, num_classes=10, learning_rate=1e-3):
        super().__init__()
        # Load EfficientNetb0 model from hugging face
        self.model = EfficientNetForImageClassification.from_pretrained(
            "google/efficientnet-b0",
            num_labels=num_classes,
            ignore_mismatched_sizes=True)

        self.criterion = nn.CrossEntropyLoss()
        self.accuracy = Accuracy(task="multiclass", num_classes=num_classes)
        self.learning_rate = learning_rate
        self.train_metrics = []
        self.val_metrics = []

    def forward(self, x):
        return self.model(x)

    def configure_optimizers(self):
        optimizer = torch.optim.Adam(self.parameters(), lr=self.learning_rate)
        return optimizer

|
net = LitEfficientNetV2().to(device)

```

สร้าง model โดยการไปเรียกใช้ model สำเร็จรูปที่มีชื่อว่า efficientnet-b0 ซึ่งเป็น model ของ google โดยและให้มันทำนาย class ออกมาตามจำนวน label ที่เรามี และ กำหนดให้ใช้ crossEntropyLoss เป็น loss function และกำหนด learning rate เป็น 0.001และมีการใช้ optimization แบบ Adam

```

=====
Layer (type:depth-idx)                               Output Shape           Param #
=====
LitEfficientNetV2                                     [32, 10]               --
├─EfficientNetForImageClassification: 1-1             [32, 10]               --
│   └─EfficientNetModel: 2-1                          [32, 1280]             --
│       ├──EfficientNetEmbeddings: 3-1                 [32, 32, 112, 112]     928
│       ├──EfficientNetEncoder: 3-2                   [32, 1280, 7, 7]       4,006,620
│       └─AvgPool2d: 3-3                               [32, 1280, 1, 1]       --
│   └─Dropout: 2-2                                     [32, 1280]             --
│       └─Linear: 2-3                                  [32, 10]               12,810
=====

```

นี่คือ layer ที่ efficientnet-b0 defined ไว้จะเห็นว่ามี param สูงถึง 4 ล้านหกแสนตัว และมีการใช้ AvgPool2d แทน maxPool2d ที่มักใช้กับ CNN และมีการใช้ drop out เพื่อตัดผลการใช้งานผลลัพธ์จาก บาง node และใช้ fully connected ในตอนสุดท้าย



|   | Name      | Type                               | Params | Mode  | FLOPs |
|---|-----------|------------------------------------|--------|-------|-------|
| 0 | model     | EfficientNetForImageClassification | 4.0 M  | train | 0     |
| 1 | criterion | CrossEntropyLoss                   | 0      | train | 0     |
| 2 | accuracy  | MulticlassAccuracy                 | 0      | train | 0     |

Trainable params: 4.0 M  
 Non-trainable params: 0  
 Total params: 4.0 M  
 Total estimated model params size (MB): 16  
 Modules in train mode: 333  
 Modules in eval mode: 0  
 Total FLOPs: 0

ผลของการ train มี 4 ล้าน parameter และ ขนาดของ model param เป็น 16Mb

```

.. testing ...
100% ██████████ 10/10 [00:02<00:00, 3.87it/s]
testing loss: 0.285

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.7500    | 1.0000 | 0.8571   | 30      |
| 1            | 0.8788    | 0.9667 | 0.9206   | 30      |
| 2            | 0.9062    | 0.9667 | 0.9355   | 30      |
| 3            | 0.9032    | 0.9333 | 0.9180   | 30      |
| 4            | 0.9231    | 0.8000 | 0.8571   | 30      |
| 5            | 0.9667    | 0.9667 | 0.9667   | 30      |
| 6            | 1.0000    | 0.8333 | 0.9091   | 30      |
| 7            | 0.9643    | 0.9000 | 0.9310   | 30      |
| 8            | 1.0000    | 0.8667 | 0.9286   | 30      |
| 9            | 0.9310    | 0.9000 | 0.9153   | 30      |
| accuracy     |           |        | 0.9133   | 300     |
| macro avg    | 0.9223    | 0.9133 | 0.9139   | 300     |
| weighted avg | 0.9223    | 0.9133 | 0.9139   | 300     |

เมื่อเอามา test กับ test set จะเห็นว่าตอนนี้ class 3 คะแนนพุ่งขึ้นมา ดีกว่า 2.1 เสียอีก และได้ผล macro\_avg f1\_score เป็น 0.9139 และยิ่งไปกว่านั้น precission ของ class 6, 8 ได้คะแนนเต็มซึ่ง ทุกครั้งที่ทายว่าเป็น class 6,8 นั้นเป็น class ดังกล่าวจริงๆทุกครั้ง

## 5. Image classification with ViT from Hugging Face & TensorBoard

```
from datasets import load_dataset
ds = load_dataset('beans',)
ds
```

ทำการ Load datasets

```
from transformers import ViTImageProcessor

model_name_or_path = 'google/vit-base-patch16-224'
feature_extractor = ViTImageProcessor.from_pretrained(model_name_or_path)
```

ทำการสร้าง feature extractor (จริงๆควรตั้งชื่อ ว่า transformer เพราะเราทำ transform data) ออกมาเพื่อ transform data โดยการ Load model vit-base-patch16-224 จาก transformers โดยใช้ model ที่ถูก train มาแล้ว และเป็น model แบบ default จาก transformers เพื่อเอา transform ของ default model มาเพราะเราจะได้ไม่ต้องสร้าง transformation เอง

```
feature_extractor

... ViTImageProcessor {
  "do_convert_rgb": null,
  "do_normalize": true,
  "do_rescale": true,
  "do_resize": true,
  "image_mean": [
    0.5,
    0.5,
    0.5
  ],
  "image_processor_type": "ViTImageProcessor",
  "image_std": [
    0.5,
    0.5,
    0.5
  ],
  "resample": 2,
  "rescale_factor": 0.00392156862745098,
  "size": {
    "height": 224,
    "width": 224
  }
}
```

นี่คือ configuration ของ model ที่มีการทำ Preprocessing เช่นทำให้ภาพเป็นขนาด 224 x 224 และ normalize ค่าสีของ RGB ให้เป็นตั้งแต่ 0-1 และค่าสีกระจายๆเฉลี่ยอยู่ที่ 0.5

```

def transform(example_batch):
    # Take a list of PIL images and turn them to pixel values
    inputs = feature_extractor([x for x in example_batch['image']], return_tensors='pt')
    # Don't forget to include the labels!
    inputs['labels'] = example_batch['labels']
    return inputs

prepared_ds = ds.with_transform(transform)

```

ทำการ **defined function** ให้ **dataset** ที่ก่อนจะถูกนำไปใช้ต้องเรียกฟังก์ชัน **Transform** ที่ไปเรียก **default transformation** ของ **default model** ก่อนอีกที

```

from transformers import ViTForImageClassification

labels = ds['train'].features['labels'].names
model = ViTForImageClassification.from_pretrained('google/vit-base-patch16-224',
                                                    num_labels=len(labels),
                                                    ignore_mismatched_sizes=True)

```

สร้าง **model** จาก **vitb-base-path16-224** โดยที่ **class** ที่ออกมาทั้งหมดจะต้องเท่ากับ จำนวน **labels** ของ **datasets** เรา

```

from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="vit-base-beans-demo-v5",
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_strategy='epoch',
    logging_steps=1,
    num_train_epochs=10,
    optim='adamw_torch',
    learning_rate=1e-5,
    remove_unused_columns=False,
    report_to='tensorboard',
    load_best_model_at_end=True,
    metric_for_best_model="loss",
    seed=0
)

```

กำหนด **argument** ของ **model** เช่น **epochs = 10**, **optimize** ให้ **adam**, **learning rate = 0.00001**, ใช้  
 การเลือก **model** จาก **loss** ที่น้อยที่สุด กำหนด **batch size** ของ **train** และ **evaluate** เป็น **16** และกำหนด **seed** เป็น **0**  
 เพื่อให้ การ **train** แต่ละครั้งได้ผลที่สามารถเทียบกันได้ เช่น ถ้าจะปรับ **parameter** แล้วการจะได้ผลดีกว่า ก็ต้องเทียบ  
 กับการสุ่ม **data** ที่เอามา **train** เดียวกัน



```
from transformers import Trainer
```

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    data_collator=collate_fn,  
    compute_metrics=compute_metrics,  
    train_dataset=prepared_ds["train"],  
    eval_dataset=prepared_ds["validation"],  
    processing_class=feature_extractor,  
)
```

ทำการ สร้าง trainer โดยใช้ model , argument ต่างๆ และ datasets ที่เราเตรียมไว้ให้ พร้อมทั้งกำหนด transformation ให้ เพื่อให้คนอื่นที่อาจจะโหลด model ไปใช้ไม่ต้อง defined transformation เอง

| ...               | precision | recall | f1-score | support |
|-------------------|-----------|--------|----------|---------|
| angular_leaf_spot | 0.2264    | 0.2791 | 0.2500   | 43      |
| bean_rust         | 0.0000    | 0.0000 | 0.0000   | 43      |
| healthy           | 0.2703    | 0.4762 | 0.3448   | 42      |
| accuracy          |           |        | 0.2500   | 128     |
| macro avg         | 0.1656    | 0.2518 | 0.1983   | 128     |
| weighted avg      | 0.1647    | 0.2500 | 0.1971   | 128     |

ผลลัพธ์ก่อน fine-tune จะเห็นว่า macro\_avg f1\_score ต่ำมาก

```
***** train metrics *****
```

```
epoch = 10.0  
total_flos = 746244894GF  
train_loss = 0.0768  
train_runtime = 0:09:26.63  
train_samples_per_second = 18.248  
train_steps_per_second = 1.147
```

ผลของการ train จะมี loss เพียง 0.0768 และใช้เวลาไป 9 นาที 26.63 วินาที

```
***** eval metrics *****
epoch                        =      10.0
eval_accuracy                =      0.985
eval_loss                    =      0.0256
eval_model_preparation_time =      0.0107
eval_runtime                  = 0:00:02.35
eval_samples_per_second      =     56.436
eval_steps_per_second        =      3.819
```

ผลของการ evaluate ด้วย validation set จะเห็นว่า loss เพียงแค่ 0.0256 ซึ่งน้อยกว่า train datasets อีก

| ...               | precision | recall | f1-score | support |
|-------------------|-----------|--------|----------|---------|
| angular_leaf_spot | 1.0000    | 0.9302 | 0.9639   | 43      |
| bean_rust         | 0.9149    | 1.0000 | 0.9556   | 43      |
| healthy           | 0.9756    | 0.9524 | 0.9639   | 42      |
| accuracy          |           |        | 0.9609   | 128     |
| macro avg         | 0.9635    | 0.9609 | 0.9611   | 128     |
| weighted avg      | 0.9634    | 0.9609 | 0.9611   | 128     |

ผลของการ classifier ว่าเป็นถั่วประเภทไหนได้ผลดังนี้จะเห็นว่า macro\_avg f1\_score พุ่งขึ้นมาถึง 0.9611 ทำให้การ finetune ครั้งนี้ได้ผลลัพธ์ดีขึ้นกับ dataset ของเรา

## 7\_1\_LLM\_Basic\_API\_Call\_LangChain

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-5-mini",
    temperature=0,
    max_tokens=None,
    timeout=None,
    max_retries=2,
    # api_key="...", # if you prefer to pass api key in directly instead of using env vars
)
```

เป็นการเรียกใช้ LLM ของ OpenAI ด้วยเลือก model gpt-5-mini และกำหนด Temperature ให้มันเลือกคำตอบที่ช้าที่สุดเสมอโดยการกำหนดเป็น 0

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0,
    max_tokens=None,
    timeout=None,
    max_retries=2,
    # api_key="...", # if you prefer to pass api key in directly instead of using env vars
)
```

เป็นการเรียกใช้ LLM ของ OpenAI ด้วยเลือก model gemini-2.5-flash และกำหนด Temperature ให้มันเลือกคำตอบที่ช้าที่สุดเสมอโดยการกำหนดเป็น 0

```
messages = [
    (
        "system",
        "You are a helpful assistant that translates English to French. Translate the user sentence.",
    ),
    ("human", "I love programming."),
]
ai_msg = llm.invoke(messages)
ai_msg
```

เป็น template ของ gpt ที่จะกำหนด role ของ ai เป็น system และ user เป็น human โดยเราสามารถกำหนด Priority ของ AI ให้เวลาตอบยึดสิ่งนี้ไว้ตลอดได้โดยการกำหนด พฤติกรรมของมันที่ message แรกได้ ซึ่งในที่นี้ เป็นเรากำหนดให้ เป็นผู้ช่วยแปลภาษาจาก อังกฤษเป็นฝรั่งเศส และให้เราแปลเป็นออกมา ซึ่งเราก็ให้มันแปลคำว่า “I love programming” แต่ด้วยผมไม่มี quota ดังนั้นก็จะมีตัวอย่างคำตอบให้ดู



```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0,
    max_tokens=None,
    timeout=None,
    max_retries=2,
    # api_key="...", # if you prefer to pass api key in directly instead of using env vars
)

messages = [
    (
        "system",
        "You are a helpful assistant that translates English to French. Translate the user sentence.",
    ),
    ("human", "I love programming."),
]
ai_msg = llm.invoke(messages)
ai_msg

... AIMessage(content="J'adore la programmation.", additional_kwargs={}, response_metadata={'finish_reason': 'STOP', 'model_name': 'gemini-2.5-flash', 'safety_ratings': [], 'model_provider': 'google_genai'}, id='lc_run--019c5779-5a4b-75d3-96d7-f2571db33aae-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 21, 'output_tokens': 7, 'total_tokens': 28, 'input_token_details': {'cache_read': 0}})
```

นี่คือตัวอย่างการเรียกใช้งาน gemini ai ผ่าน lang-chain โดยที่เราจะได้ คำตอบของการแปลภาษาว่า “J'adore la programmation.”

```
from langchain_groq import ChatGroq

llm = ChatGroq(
    model="llama-3.1-8b-instant", # can change
    temperature=0,
    max_tokens=None,
    timeout=None,
    max_retries=2,
)

messages = [
    (
        "system",
        "You are a helpful assistant that translates English to French. Translate the user sentence.",
    ),
    ("human", "I love programming."),
]
ai_msg = llm.invoke(messages)
ai_msg

AIMessage(content='The translation of "I love programming" to French is:\n\n"J\'adore le programmation."', additional_kwargs={}, response_metadata={'token_usage': {'completion_tokens': 22, 'prompt_tokens': 55, 'total_tokens': 77, 'completion_time': 0.039521449, 'completion_tokens_details': None, 'prompt_time': 0.002793639, 'prompt_tokens_details': None, 'queue_time': 0.005381625, 'total_time': 0.042315088}, 'model_name': 'llama-3.1-8b-instant', 'system_fingerprint': 'fp_8f8420ecd7', 'service_tier': 'on demand', 'finish_reason': 'stop', 'logprobs': None, 'model_provider': 'groq'}, id='lc_run--019c577b-267b-7061-8bb9-9796401cedb7-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 55, 'output_tokens': 22, 'total_tokens': 77})
```

นี่คือตัวอย่างการเรียก LLM ของ meta โดยใช้ model llama-31-8n-instant โดยที่ได้คำตอบเป็น “The translation of "I love programming" to French is:\n\n"J'adore le programmation."” จะเห็นว่า model ของ llama มีการเพิ่มคำอธิบาย ไม่ได้มีการ แปลคำออกมาตรงๆ เหมือนกับ gemini ของ google



```
from langchain_nvidia_ai_endpoints import ChatNVIDIA
```

```
llm = ChatNVIDIA(model="mistralai/mixtral-8x22b-instruct-v0.1")  
result = llm.invoke("Write a ballad about LangChain.")  
print(result.content)
```

... In the realm of tech, where dreams take flight,  
There lived a hero, brave and bright.  
LangChain, a hero of code and might,  
Guarding the kingdom with all his might.

In the heart of the web, where data flows,  
LangChain weaved his tales and spun his yows.  
With algorithms strong and words that glowed,  
He crafted a world, where knowledge bestowed.

From NLP to deep learning's core,  
He roamed the lands, ever exploring more.  
Through neural nets and transformers fair,  
He built a chain of knowledge, beyond compare.

The people cried for wisdom and truth,  
LangChain heard their pleas, with courage uncouth.  
He fought through the noise, the buzz, and the clutter,  
With every line of code the kingdom would utter

ตัวอย่างผลลัพธ์ของการใช้ model จาก nvidia ซึ่งมี model เยอะมากให้เราเลือกสรร